Projeto TypeScript

Projeto TypeScript configurado para desenvolvimento profissional em Linux, com suporte a:

- **TypeScript 5** em **strict mode**
- **ESM** (`"type": "module"`)
- **ESLint 9** com **Flat Config**
- **Prettier 3**
- **Vitest** para testes automatizados
- **tsx** para execução em desenvolvimento
- **VS Code** com configurações recomendadas
- **Yarn 1.22.22** como gerenciador de pacotes

Este README descreve, passo a passo, como preparar o ambiente no Linux, instalar as dependências e executar o projeto.

1. Pré-requisitos

Antes de iniciar, verifique se o sistema possui:

- Linux atualizado
- `git`
- `curl`
- `build-essential` ou equivalente
- **Node.js 22.12.0 ou superior**
- **Yarn 1.22.22**
- **Visual Studio Code** (opcional, mas recomendado)

No Ubuntu/Debian, instale os utilitários básicos com:

```
sudo apt update
sudo apt install -y git curl build-essential
```

2. Instalação do Node.js com NVM

Como o projeto exige **Node >= 22.12.0**, a forma mais segura no Linux é usar **NVM**.

2.1 Instalar o NVM

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
```

Depois, recarregue o shell:

```
source ~/.bashrc
```

Se utilizar `zsh`:

```
source ~/.zshrc
```

2.2 Instalar e ativar o Node.js 22

```
nvm install 22
nvm use 22
nvm alias default 22
```

2.3 Confirmar a versão

```
node -v
```

A saída deve indicar versão **22.12.0 ou superior**.

3. Instalação do Yarn 1.22.22

O projeto declara explicitamente:

```
"packageManager": "yarn@1.22.22"
```

Instale o Yarn Classic com:

```
npm install -g yarn@1.22.22
```

Confirme:

```
yarn -v
```

A versão esperada é:

```
1.22.22
```

4. Obtenção do projeto

Se o projeto estiver em um repositório Git:

```
git clone <REPOSITORY_URL>
cd unit1
```

Se você recebeu o projeto compactado em `.zip`, extraia-o e entre na pasta raiz:

```
unzip setup.zip
cd typescript
```

A raiz do projeto contém, entre outros, os seguintes arquivos:

```
eslint.config.mjs
vitest.config.mjs
package.json
tsconfig.json
tsconfig.build.json
tsconfig.test.json
src/
test/
.vscode/
```

5. Instalação das dependências

Na pasta raiz do projeto, execute:

```
yarn install
```

Esse comando instalará todas as dependências listadas em `package.json`, incluindo:

- TypeScript
- ESLint
- Prettier
- Vitest
- `@vitest/coverage-v8`
- `tsx`
- `rimraf`
- `tsc-alias`

6. Configuração do arquivo `.env`

O projeto já considera o uso de variáveis de ambiente e há um exemplo simples:

```
NODE_ENV=dev
```

Caso o arquivo `.env` ainda não exista, crie-o na raiz do projeto:

```
touch .env
```

Conteúdo mínimo sugerido:

```
NODE_ENV=dev
```

Observação: o arquivo `.env` está ignorado pelo Git, portanto não será versionado.

7. Abertura no Visual Studio Code

Abra o projeto na pasta raiz:

```
code .
```

O projeto já traz configurações específicas em `.vscode/settings.json` e recomendações de extensões em `.vscode/extensions.json`.

7.1 Extensões recomendadas

Instale, quando solicitado, as extensões:

- **Prettier** (`esbenp.prettier-vscode`)
- **ESLint** (`dbaeumer.vscode-eslint`)
- **Vitest** (`vitest.explorer`)
- **TypeScript Next** (`ms-vscode.vscode-typescript-next`)

7.2 Comportamentos já configurados no VS Code

Ao salvar arquivos, o ambiente está preparado para:

- formatar com Prettier
- aplicar correções automáticas do ESLint
- preferir `type imports`
- atualizar imports ao mover arquivos
- usar o TypeScript instalado no próprio projeto

8. Estrutura e finalidade dos principais arquivos

`package.json`

Define scripts, dependências, engine mínima do Node e o gerenciador de pacotes.

`tsconfig.base.json`

Centraliza as opções principais do TypeScript, incluindo:

- `strict: true`
- `moduleResolution`
- `target` moderno
- alias `@src/*`

`tsconfig.json`

Usado para verificação principal de tipos durante o desenvolvimento no editor. Deve incluir `src/**/*.ts`, `src/**/*.d.ts` e `test/**/*.ts`, além de declarar:

```
"types": ["node", "vitest/globals"]
```

`tsconfig.build.json`

Usado no processo de build, emitindo arquivos compilados em `build/`.

`tsconfig.test.json`

Usado para testes com Vitest e para validação tipada específica dos testes.

`eslint.config.mjs`

Configuração Flat Config do ESLint 9 para código-fonte, testes e arquivos de configuração.

`vitest.config.mjs`

Configuração principal do Vitest, incluindo ambiente de execução, globais e cobertura.

9. Scripts disponíveis

Abaixo estão os scripts mais relevantes do projeto.

9.1 Desenvolvimento

Executa o arquivo de entrada com `tsx`:

```
yarn dev
```

Script correspondente:

```
"dev": "tsx ./src/main.ts"
```

9.2 Build de produção

Remove a pasta anterior, compila o projeto e ajusta aliases:

```
yarn build
```

Saída gerada em:

```
build/
```

9.3 Execução do build

Após compilar, execute:

```
yarn start
```

9.4 Verificação de tipos

Executa checagem de tipos tanto no código de produção quanto nos testes:

```
yarn type:check
```

9.5 Lint

Executa o ESLint com cache:

```
yarn lint
```

Modo estrito para integração contínua:

```
yarn lint:ci
```

9.6 Formatação

Formatar todo o projeto:

```
yarn format
```

Apenas verificar se está formatado:

```
yarn format:check
```

9.7 Testes

Executar testes:

```
yarn test
```

Executar em modo watch:

```
yarn test:watch
```

Executar com cobertura:

```
yarn test:coverage
```

Executar em modo CI, com cobertura e thresholds:

```
yarn test:ci
```

9.8 Validação completa do projeto

Para validar praticamente todo o ambiente local:

```
yarn check
```

Para o fluxo de integração contínua:

```
yarn check:ci
```

Esses scripts encadeiam:

- formatação
- lint
- type check
- testes
- cobertura

10. Fluxo recomendado de uso no Linux

Após instalar tudo, o fluxo mais seguro é:

10.1 Instalar dependências

```
yarn install
```

10.2 Validar o projeto

```
yarn check
```

10.3 Rodar o ambiente de desenvolvimento

```
yarn dev
```

10.4 Antes de finalizar alterações

```
yarn check:ci
```

Esse procedimento ajuda a garantir conformidade de estilo, tipagem e testes antes de versionar o código.

11. Exemplo de primeira execução

Sequência completa, a partir da pasta do projeto:

```
yarn install
yarn type:check
yarn lint
yarn test
yarn dev
```

Se desejar validar tudo de uma só vez:

```
yarn check
```

12. Possíveis problemas e correções

12.1 `node: command not found`

O Node.js não está instalado ou não foi carregado no shell atual.

Solução:

```
source ~/.bashrc
nvm use 22
```

12.2 `yarn: command not found`

O Yarn não foi instalado globalmente.

Solução:

```
npm install -g yarn@1.22.22
```

12.3 Erros de versão do Node

Se o projeto acusar incompatibilidade de engine, confira:

```
node -v
```

Se necessário:

```
nvm install 22
nvm use 22
```

12.4 O VS Code não reconhece `describe`, `test` ou `expect`

Esse problema costuma ocorrer quando o TypeScript do editor não está enxergando os globais do Vitest.

Verifique:

1. se `vitest/globals` está declarado em `compilerOptions.types`
2. se `test/**/*.ts` está incluído no `tsconfig.json` usado pelo editor
3. se `globals: true` está definido em `vitest.config.mjs`

Exemplo esperado no `tsconfig.json`:

```
{
  "compilerOptions": {
    "types": ["node", "vitest/globals"]
  },
  "include": ["src/**/*.ts", "src/**/*.d.ts", "test/**/*.ts"]
}
```

Depois, reinicie o servidor TypeScript no VS Code:

```
TypeScript: Restart TS Server
```

12.5 O VS Code não reconhece o TypeScript do projeto

Verifique se a pasta `node_modules` foi instalada e se o projeto foi aberto na raiz correta.

Depois, no VS Code, confirme o uso da versão local do TypeScript.

12.6 Falhas de lint ou formatação

Corrija automaticamente com:

```
yarn format
yarn lint
```

E depois revalide:

```
yarn check
```

13. Observações técnicas relevantes

- O projeto utiliza **ES Modules**, portanto imports e execução seguem esse modelo.
- O build é gerado em `build/` , não em `dist/` .
- O alias `@src/*` está configurado no TypeScript e ajustado no pós-build com `tsc-alias` .

- Os testes estão em `test/**/*.spec.ts` ou `test/**/*.test.ts`.
- O projeto utiliza **Vitest** com `globals: true`, o que permite usar `describe`, `test` e `expect` sem imports explícitos.
- A cobertura é gerada pelo provider **V8**.
- O projeto adota formatação com:
  - sem ponto e vírgula
  - aspas simples
  - vírgula trailing
  - largura máxima de linha de 120 colunas

14. Comandos essenciais de referência rápida

```
# instalar dependências
yarn install

# desenvolvimento
yarn dev

# build
yarn build

# executar build
yarn start

# verificar tipos
yarn type:check

# lint
yarn lint

# formatar
yarn format

# testes
yarn test

# cobertura
yarn test:coverage

# validação completa
yarn check

# validação CI
yarn check:ci
```

15. Conclusão

Com os passos acima, o projeto fica corretamente preparado para uso em Linux, com ambiente coerente para desenvolvimento, teste, lint, formatação e build. A combinação de **Node 22 + Yarn**

1.22.22 + VS Code + ESLint + Prettier + Vitest fornece uma base sólida para evolução do código com previsibilidade e rigor técnico.